Design proposal

Michał Podgajny, Miłosz Andruczyk, Aleksander Szymczyk

18.03.2026r.

1 Opis zadania

Projekt polega na wytrenowaniu klasyfikatora gatunku na wybranych datasetach MIDI i zastosowaniu metod concept-based interpretability do analizy, które cechy model uznaje za charakterystyczne dla każdego gatunku. Inspiracją jest praca Foscarina et al. (2022) [5], gdzie TCAV zastosowano do klasyfikacji kompozytorów. Projekt przenosi to podejście na gatunki, definiując odpowiednie koncepty muzyczne. Dodatkowym elementem jest porównanie "definicji gatunku"zakodowanych w różnych datasetach oraz analiza próbek błędnie sklasyfikowanych.

2 Harmonogram projektu

Tygodnie	Działanie	Przewidywana data ukończenia	Estymowany czas treningu modeli (GPU h)	Uwagi
1	Stworzenie design proposal oraz gotowe środowisko developerskie/eksperymentalne	18.03	–	
2	Stworzenie pipeline'u do wstępnego przetwarzania danych symbolicznych dla wybranych datasetów	25.03	–	
3	Baselinowy model klasyfikujący gatunki (klasyczny ML) oraz wstępny trening modeli neuronowych	01.04	10–20	deadline na zgłoszenie gotowości prototypu
4	Implementacja oraz trening modeli klasyfikujących z literatury	08.04	20–50	krótszy tydzień - Wielkanoc
5	Ewaluacja dotychczasowych rezultatów. Ablacje elementów architektury modeli	15.04	10–20	
6	Implementacja metody TCAV na klasyfikatorze osiągającym najlepsze rezultaty	22.04	20–50	
7	Analiza wyników interpretowalności oraz wizualizacja wpływu konceptów na predykcje modelu	29.04	5–10	
8	Porównanie definicji gatunku pomiędzy różnymi datasetami MIDI	06.05	4–10	
9	Analiza błędnie sklasyfikowanych próbek oraz identyfikacja potencjalnych przyczyn błędów	13.05	–	
10	Integracja wszystkich elementów projektu oraz przygotowanie wizualizacji wyników	20.05	–	
11	Nagranie filmiku, ostateczne poprawki dokumentacji i przygotowanie prezentacji	25.05	–	deadline na oddanie projektu ze zwolnieniem z kolokwium

2.1 Przewidywane zużycie zasobów

Do realizacji projektu wystarczająca powinna być karta graficzna wyposażona w 12–20 GB VRAM. Szacowany koszt obliczeniowy wynosi około 80–160 GPU h, przy założeniu wydajności odpowiadającej karcie NVIDIA A100 lub jednostce o zbliżonej mocy obliczeniowej.

3 Stack technologiczny

Do przetwarzania plików MIDI zdecydowaliśmy się na wybór biblioteki partitura, ponieważ Foscarin et al. (2022) [5] używa jej w swojej pracy. Ponadto jest ona częściej aktualizowana niż pretty_midi i miditoolkit.

3.1 Warstwa badawczo-eksperymentalna

Technologia	Zastosowanie
pandas	Przetwarzanie danych tabelarycznych oraz agregacja wyników eksperymentów
partitura	Przetwarzanie plików MIDI oraz ekstrakcja reprezentacji symbolicznych i cech muzycznych wykorzystywanych w dalszej analizie
PyTorch	Implementacja, trenowanie i ewaluacja modeli sieci neuronowych do klasyfikacji gatunku muzycznego
scikit-learn	Analiza wyników, obliczanie metryk ewaluacyjnych oraz implementacja modeli bazowych
Captum	Analiza interpretowalności modelu, w tym zastosowanie metod concept-based interpretability, takich jak TCAV
matplotlib	Wizualizacja wyników eksperymentów oraz rezultatów analizy interpretowalności

3.2 Warstwa inżyniersko-organizacyjna

Technologia	Zastosowanie
Hydra	Zarządzanie konfiguracją eksperymentów, hiperparametrami oraz wariantami uruchomień
Weights & Biases (wandb)	Monitorowanie eksperymentów oraz porównywanie wyników między uruchomieniami
Ruff	Statyczna analiza kodu oraz automatyczne formatowanie zgodne z przyjętym stylem projektu
pytest	Automatyczne testy poprawności środowiska, komponentów projektu oraz pipeline'u eksperymentalnego
uv	Zarządzanie środowiskiem wirtualnym i zależnościami projektu na podstawie pliku `pyproject.toml`
Make	Oskryptowane uruchamianie najważniejszych zadań projektu, takich jak instalacja zależności, testy i linting
MkDocs	Przygotowanie dokumentacji projektu i instrukcji użytkowania

4 Zakres eksperymentów

Celem eksperymentów jest ocena jakości modeli klasyfikujących gatunki muzyczne na podstawie danych MIDI oraz analiza interpretowalności ich predykcji z wykorzystaniem metod concept-based interpretability.

4.1 Modele bazowe i modele sieci neuronowych

W pierwszym etapie zostaną zaimplementowane modele bazowe oraz modele bardziej złożone:

- Modele klasyczne(baseline):
  - Logisitc Regression
  - Random Forest
- Modele neuronowe:
  - MLP na ręcznie wyeakstrahowanych cechach
  - modele sekwencyjne (np. Transformer) operujące bezpośrednio na reprezentacji MIDI

Celem jest określenie, jak bardzo modele głębokie przewyższają klasyczne podejścia oraz które reprezentacje danych są najbardziej informatywne.

4.2 Ablacja architektury modeli

Dla najlepszego modelu neuronowego zostaną przeprowadzone eksperymenty mające na celu analizę wpływu poszczególnych komponentów architektury na jakość klasyfikacji. Rozważone zostaną następujące czynniki:

- złożoność modelu: zmiana liczby warstw oraz wymiaru embeddingów,
- regularyzacja: wpływ dropout oraz innych technik ograniczających przeuczenie,
- długość sekwencji wejściowej: analiza wpływu ilości kontekstu czasowego na jakość predykcji.

Celem jest zrozumienie, które komponenty architektury mają największy wpływ na jakość predykcji.

4.3 Ewaluacja modeli

Modele będą oceniane przy użyciu standardowych metryk klasyfikacyjnych:

- accuracy,
- precision, recall, F1-score (macro i weighted),
- macierz pomyłek umożliwiająca analizę najczęściej mylonych gatunków.

4.4 Eksperymenty interpretowalności (TCAV)

Kluczowym elementem projektu jest analiza interpretowalności modeli z użyciem TCAV. Planowane kroki:

- zdefiniowanie konceptów muzycznych (np. rytmiczność, złożoność harmoniczna, tempo, repetetywność),
- przygotowanie zbiorów przykładów reprezentujących dany koncept,
- obliczenie TCAV scores dla poszczególnych klas (gatunków).

Celem jest określenie, które koncepty mają największy wpływ na decyzje modelu oraz czy są one zgodne z intuicją muzyczną.

4.5 Porównanie datasetów

Zostanie przeprowadzona analiza różnic pomiędzy datasetami MIDI:

- trenowanie modelu na jednym datasetcie i testowanie na innym,
- porównanie wyników TCAV między datasetami.

Celem jest sprawdzenie, czy modele uczą się uniwersalnych cech gatunkowych, czy raczej specyficznych dla datasetu.

4.6 Analiza błędów

Dla błędnie sklasyfikowanych próbek zostanie wykonana szczegółowa analiza:

- identyfikacja najczęściej mylonych gatunków,
- analiza struktury muzycznej błędnych przykładów,
- porównanie wartości TCAV dla poprawnych i błędnych predykcji.

Celem jest zrozumienie ograniczeń modelu oraz identyfikacja potencjalnych kierunków poprawy.

5 Funkcjonalność programu

Tworzony system będzie miał charakter narzędzia badawczo-eksperymentalnego wspierającego klasyfikację gatunków muzycznych na podstawie plików MIDI oraz analizę interpretowalności uzyskanych modeli. Przewidywane funkcjonalności programu obejmują:

1. wczytywanie i walidację plików MIDI pochodzących z wybranych datasetów,
2. wstępne przetwarzanie danych symbolicznych oraz ich konwersję do wspólnej reprezentacji wykorzystywanej przez modele,
3. ekstrakcję reprezentacji i cech muzycznych używanych w klasyfikacji oraz analizie interpretowalności,
4. trenowanie modeli bazowych oraz modeli sieci neuronowych do klasyfikacji gatunku muzycznego,
5. ewaluację modeli z użyciem standardowych metryk klasyfikacyjnych oraz generowanie macierzy pomyłek,
6. zarządzanie konfiguracją eksperymentów i rejestrowanie ich wyników w sposób reprodukowalny,
7. zastosowanie metod concept-based interpretability, w szczególności TCAV, do analizy wpływu wybranych konceptów muzycznych na predykcje modelu,
8. analizę błędnie sklasyfikowanych próbek oraz identyfikację potencjalnych przyczyn błędów,
9. porównanie wyników uzyskanych na różnych datasetach MIDI, w tym analizę różnic w zakodowanych w nich cechach gatunkowych,
10. generowanie wykresów, tabel oraz podsumowań wyników na potrzeby dokumentacji i prezentacji projektu.

Program będzie uruchamiany z poziomu linii poleceń i konfigurowany za pomocą plików konfiguracyjnych, co pozwoli na wygodne odtwarzanie eksperymentów oraz porównywanie różnych wariantów modeli i ustawień.

Literatura

[1] Kjell Lemström Andres Ferraro. On large-scale genre classification in symbolically encoded music by automatic identification of repeating patterns.

[2] Carlos Eduardo Cancino-Chacón, Silvan David Peter, Emmanouil Karystinaios, Francesco Foscarin, Maarten Grachten, and Gerhard Widmer. Partitura: A Python Package for Symbolic Music Processing. In *Proceedings of the Music Encoding Conference (MEC2022)*, Halifax, Canada, 2022.

[3] Edmund Dervakos, Natalia Kotsani, and Giorgos Stamou. Genre recognition from symbolic music with cnns: Performance and explainability. *SN Computer Science*, 4(2):106, 2023.

[4] Andres Ferraro and Kjell Lemström. On large-scale genre classification in symbolically encoded music by automatic identification of repeating patterns. In *Proceedings of the 5th International Conference on Digital Libraries for Musicology*, DLfM '18, page 34–37. ACM, September 2018.

[5] Francesco Foscarin, Katharina Hoedt, Verena Praher, Arthur Flexer, and Gerhard Widmer. Concept-based techniques for "musicologist-friendly" explanations in a deep music classifier. In *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR)*, pages 876–883, 2022.

[6] Emmanouil Karystinaios, Francesco Foscarin, and Gerhard Widmer. Smug-explain: A framework for symbolic music graph explanations. *CoRR*, abs/2405.09241, 2024.

[7] Narine Kokhlikyan, Vivek Miglani, Miguel Martin, Edward Wang, Bilal Alsallakh, Jonathan Reynolds, Alexander Melnikov, Natalia Kliushkina, Carlos Araya, Siqi Yan, and Orion Reblitz-Richardson. Captum: A unified and generic model interpretability library for pytorch, 2020.

[8] Adi Ruppin and Hezy Yeshurun. Midi music genre classification by invariant features. pages 397–399, 01 2006.